- *Generated fields and database-computed values*—Django 5 added the ability to perform even more computations in the database. In a field, the new `db_default` argument enables computation of a value when a row gets created. For example, the database can determine and store the current time. Examples for this field are found here: https://mng.bz/MZ12.
- The other new addition is the `GeneratedField` class, which is used to compute a field's value based on the combination of other fields. This is done in the database, meaning the object doesn't have to be brought into Django's memory to determine its value. The mechanism allows both stored values (computed and stuck in the table) and virtual values (computed on the fly). Details on the `GeneratedField` class can be found here: https://mng.bz/aEVJ. Both of these features are highly dependent on your database's capabilities.
- *Syndication feed framework*—RSS and Atom feeds are API-like summaries of the content on your site, meant to be consumed by users or tools. When looking for the lastest episode of my favorite podcasts, I read the site's RSS feed rather than dig through all the "fancy-shmancy" HTML. Django has a class to build these kinds of feeds, called `Feed`. To build a `Feed` class, you tie a query to the values you want in the feed and register it as a route. You can learn more about syndication here: https://mng.bz/gvAR.
- *System check framework*—When you start the development server, Django runs a check on your project. You can invoke the same process with the `check` management command. Django allows you to register your own classes to be run as part of the check process, giving you a hook on code validation or other activities. Instructions on how to write and register your own checks are provided here: https://docs.djangoproject.com/en/dev/topics/checks/.
- *Generic foreign keys*—In chapter 4, you learned about foreign key relations and many-to-many relations, but Django has one more relationship type. Django tracks the kinds of `Model` classes in your system using something called the *content types framework*. This framework enables you to build a foreign key that relates to more than one kind of `Model`. The `GenericForeignKey` field tracks the ID of a `Model` class's content type and the specific object ID. Consider a media database with `Audio`, `Video`, and `Text` `Model` classes. The cross-media top-10 list could be built using a `ListItem` `Model`. This `Model` would use a generic foreign key that could point to any of the three media types. An instance of `ListItem` points to a `Song` by using the content type ID of the `Song` class together with the ID of the `Song` object instance. Details on the content type framework can be found in the documentation: https://mng.bz/Ddp0.

14.2 Configuration

The use of settings.py is a bit limiting in modern engineering environments. Commercial-grade systems tend to be developed using multiple environments: development, testing, staging, and production. The need for different configurations in different